

EE2C2 Mixed-Signal Circuits and Systems

Lab 3: Digital Circuits on FPGA

Chang Gao*

June 9, 2026

How to use this handout

This handout is a **step-by-step tutorial** organised into **topics**. Each topic has a **Background** part (the concept) followed by a **Lab** part (exact actions). For most actions you are given two or three interchangeable ways to do the same thing:

GUI

Click this path of menus/buttons in Vivado. Breadcrumbs are written $\mathbf{A} \rightarrow \mathbf{B} \rightarrow \mathbf{C}$.

PowerShell

Type this in a Windows PowerShell window (dark code block).

Tcl Console

Type this in the **Tcl Console** at the bottom of the Vivado window (green code block).

Figure — each figure uses a normal `\includegraphics[width=...]{...}` line and a standard numbered caption (**Figure 1**, **Figure 2**, ...). Save your capture at the path shown in the source and adjust that figure's width or height directly.

Checkpoint (yellow box) — a short question; write the answer in your report worksheet.

Note (orange box) — a pitfall or important detail.

Note

The GUI and the command line do the same thing. Vivado is scriptable: every time you click a button, it prints the equivalent **Tcl** command in the Tcl Console. Watching that console is the fastest way to learn the commands below. PowerShell simply runs Vivado in batch mode (`-mode batch`), i.e. it runs a Tcl script with no window.

Note

AUP-ZU3 box contents and return. The AUP-ZU3 box contains one FPGA board, one USB-C-to-USB-C cable for power, one USB-A-to-USB-C cable for the PC connection, and one power adapter. Please keep all items together and return the complete box after the lab.

*Email: chang.gao@tudelft.nl

1 Topic 0 — Getting started: Vivado, the design flow, and the project

1.1 Background

A **Field-Programmable Gate Array (FPGA)** is a chip full of configurable look-up tables, flip-flops, wiring, and I/O. You describe hardware in **SystemVerilog**; the tool configures the FPGA to *become* that hardware. **Vivado** is the design tool. The path from source to a running board is the **design flow**:

```
SystemVerilog (RTL) --simulate--> Synthesis (RTL -> LUT/FF netlist)
--> Implementation (place & route) --> Bitstream (.bit) --> Program over JTAG
```

You will walk the tool flow in Topics 1–7 using an **AC701 / Artix-7** project, because that board and device support are installed on the university PCs by default. The physical lab board on the bench is the **RealDigital AUP-ZU3-4GB**; the AUP-ZU3 demos in Topics 8–9 are programmed from **prebuilt bitstreams** so you can still see the LED behaviour even if the campus Vivado installation does not include XCZU3EG synthesis/implementation support.

1.2 Lab 0: clone the repo, create the project, and tour Vivado

Step 0.1 — Clone the lab repository and open it in PowerShell.

The lab lives on GitHub at https://github.com/lab-emi/EE2C2_FPGA_Lab.

PowerShell open Windows PowerShell, go to your home folder with `cd ~` (PowerShell expands `~` to `C:\Users\<you>`), then clone and enter it:

```
cd ~ # your user home folder, e.g. C:\Users\<you>
git clone https://github.com/lab-emi/EE2C2_FPGA_Lab.git
cd EE2C2_FPGA_Lab
dir
```

You should now see `build.tcl`, `prebuilt_bitstreams`, and the `src`, `sim`, `constr`, `scripts` folders (now under `C:\Users\<you>\EE2C2_FPGA_Lab`).

Note

No Git? On the GitHub page choose **Code** → **Download ZIP** and extract it; the rest of the lab is identical once you `cd` into the extracted folder.

Add Vivado and Vitis to the PATH for this PowerShell window, so you can type `vivado` instead of its full path:

```
$env:Path += ";C:\Xilinx\Vivado\2023.2\bin"
$env:Path += ";C:\Xilinx\Vitis\2023.2\bin"
vivado -version
```

`vivado -version` should print `Vivado v2023.2`. From now on this handout writes `vivado` ... instead of the full path, and assumes your prompt is **inside the cloned EE2C2_FPGA_Lab folder**.

Note

Still says vivado is not recognised? Check whether Vivado is installed in a different folder, then replace the path above with the correct `bin` folder.

```

PS C:\WINDOWS\system32> cd ~
PS C:\Users\cgao> git clone https://github.com/lab-emi/EE2C2_FPGA_Lab.git
Cloning into 'EE2C2_FPGA_Lab'...
remote: Enumerating objects: 91, done.
remote: Counting objects: 100% (91/91), done.
remote: Compressing objects: 100% (64/64), done.
remote: Total 91 (delta 17), reused 86 (delta 15), pack-reused 0 (from 0)
Receiving objects: 100% (91/91), 3.62 MiB | 14.44 MiB/s, done.
Resolving deltas: 100% (17/17), done.
PS C:\Users\cgao> cd .\EE2C2_FPGA_Lab\
PS C:\Users\cgao\EE2C2_FPGA_Lab> dir

        Directory: C:\Users\cgao\EE2C2_FPGA_Lab

Mode                LastWriteTime         Length Name
----                -
d-----          2026/5/31      18:50          board-files
d-----          2026/5/31      18:50          constr
d-----          2026/5/31      18:50        constraints
d-----          2026/5/31      18:50        lab_handout
d-----          2026/5/31      18:50    legacy_rainbow_src
d-----          2026/5/31      18:50    optional_pwm_dac
d-----          2026/5/31      18:50        scripts
d-----          2026/5/31      18:50          sim
d-----          2026/5/31      18:50          src
-a-----          2026/5/31      18:50        1849 .gitignore
-a-----          2026/5/31      18:50        3104 build.tcl
-a-----          2026/5/31      18:50        1121 LICENSE
-a-----          2026/5/31      18:50        1742 program_jtag.tcl
-a-----          2026/5/31      18:50        4464 README.md

PS C:\Users\cgao\EE2C2_FPGA_Lab> $env:Path += ";C:\Xilinx\Vivado\2023.2\bin"
PS C:\Users\cgao\EE2C2_FPGA_Lab> vivado -version
ECHO is off.
ECHO is off.
vivado v2023.2 (64-bit)
Tool Version Limit: 2023.10
SW Build 4029153 on Fri Oct 13 20:14:34 MDT 2023
IP Build 4028589 on Sat Oct 14 00:45:43 MDT 2023
SharedData Build 4025554 on Tue Oct 10 17:18:54 MDT 2023
Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
Copyright 2022-2023 Advanced Micro Devices, Inc. All Rights Reserved.

```

Figure 1: PowerShell ready in the cloned lab folder.

Step 0.2 — Create the Vivado project.

PowerShell this builds the project without synthesis (fast):

```
vivado -mode batch -source scripts\create_vivado_project.tcl
```

Watch for INFO: Using AC701 implementation part: xc7a200tfbg676-2 and INFO: Created project: ...\\build\\EE2C2_FPGA_Lab.xpr. This project targets the AC701 board part xilinx.com:ac701:part it is the project you use for simulation setup, synthesis, implementation, timing, and power reports.

Note

create_vivado_project.tcl deletes and recreates the build\\ folder every run. That is fine — all sources live in src\\, sim\\, constr\\, never in build\\.

```
INFO: Created project: C:/Users/cgao/EE2C2_FPGA_Lab/build/EE2C2_FPGA_Lab.xpr
INFO: [Common 17-206] Exiting Vivado at Sun May 31 19:34:26 2026...
PS C:\Users\cgao\EE2C2_FPGA_Lab>
```

Figure 2: Project created from the Tcl script.

Step 0.3 — Open the project in the GUI.

PowerShell

```
vivado build\\EE2C2_FPGA_Lab.xpr
```

GUI (alternative) Start menu → **Vivado 2023.2** → **Open Project** → select build\\EE2C2_FPGA_Lab.xpr.

Find the four areas you will use constantly: **Flow Navigator** (left), **Sources** (centre), **Tcl Console** (bottom), and **Messages** (bottom). Try typing version -short in the Tcl Console.

Step 0.4 — Find the source files.

GUI In **Sources**, expand **Design Sources** (top_ac701, comb_led_logic, one_hz_tick, led_fsm), **Constraints** (ac701_lab.xdc), and **Simulation Sources** (tb_comb_led_logic, tb_led_fsm).

Tcl Console (alternative)

```
report_compile_order -fileset sources_1
get_files
```

Checkpoint — answer in the report worksheet

Checkpoint 0. Confirm the project was created and record your Vivado version (2023.2).

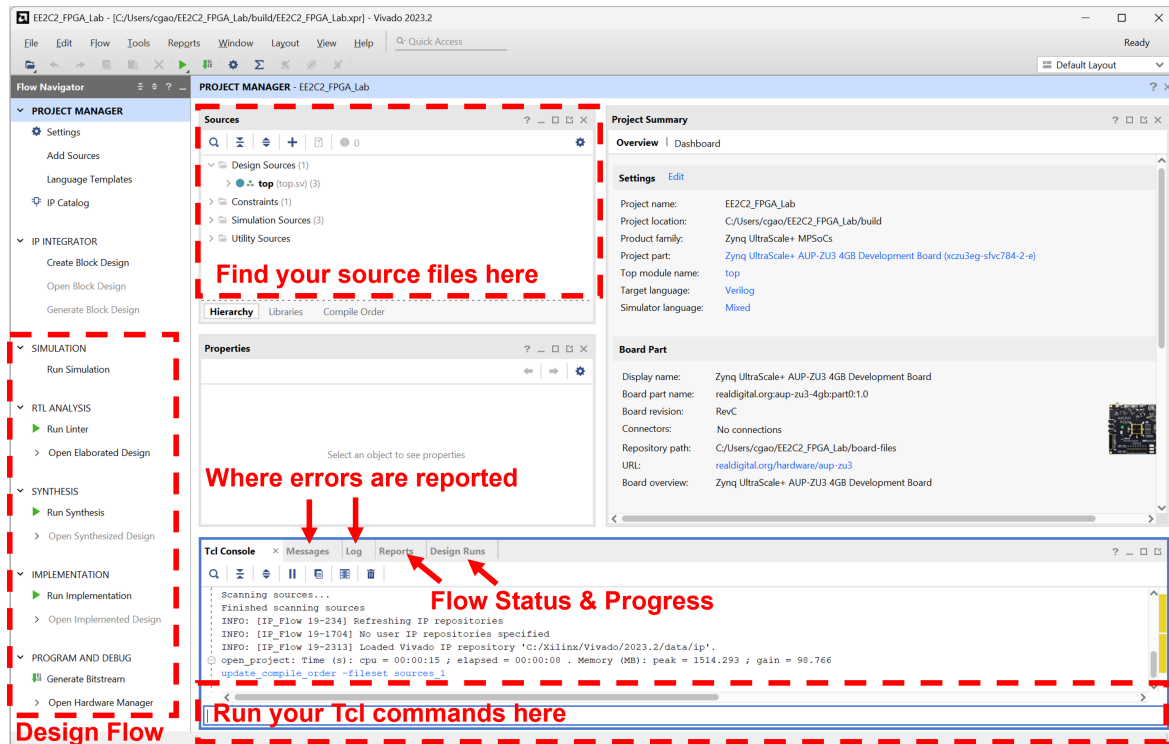


Figure 3: The Vivado main window.

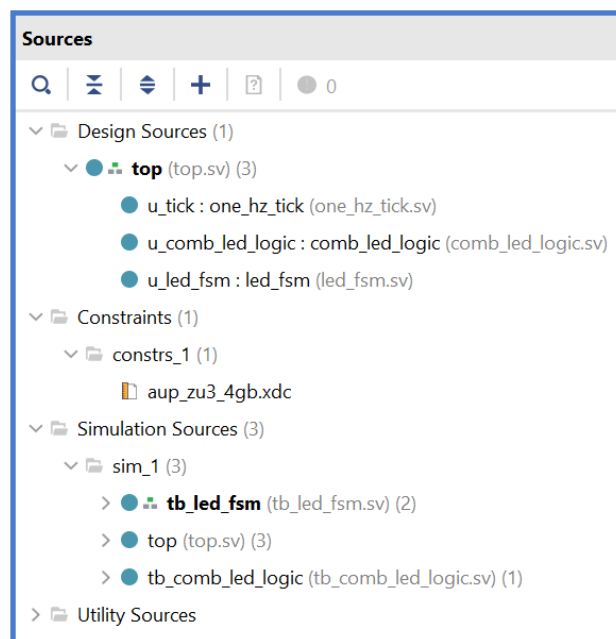


Figure 4: The Sources panel, fully expanded.

2 Topic 1 — Combinational logic

2.1 Background

Combinational logic produces outputs that depend *only on the current inputs* — no memory. Examples: gates, multiplexers, decoders, comparators. In SystemVerilog you use `assign` or an `always_comb` block:

```
assign y = (a & b) | (~a & c);
```

This does not “execute” like software — it *describes hardware that is always present*. When `a`, `b`, or `c` changes, `y` settles after the gate delay.

Note

Avoid latches. Inside an `always_comb`, assign every output on every path (the lab code writes a default value first). An unassigned output infers an unwanted memory element (a latch).

2.2 Lab 1: inspect and simulate the combinational logic

Step 1.1 — Open the source.

GUI **Sources** → **Design Sources** → double-click `comb_led_logic`. Read the `always_comb` block; note the default `led_comb = 8'b0000_0000`; that prevents latches.

```

9 module comb_led_logic (
10     input logic [3:0] sw,
11     input logic      btn,
12     output logic [7:0] led_comb,
13     output logic [2:0] rgb_comb
14 );
15
16 always_comb begin
17     led_comb = 8'b0000_0000;
18
19     led_comb[0] = sw[0];
20     led_comb[1] = sw[0] & sw[1];
21     led_comb[2] = sw[0] ^ sw[1];
22     led_comb[3] = sw[2] ? sw[3] : sw[0];
23     led_comb[4] = ~sw[0];
24     led_comb[5] = btn;
25     led_comb[6] = sw[2] | sw[3];
26     led_comb[7] = &sw;
27
28     // Simple RGB decode: {red, green, blue}.
29     unique case (sw[1:0])
30         2'b00: rgb_comb = 3'b000;
31         2'b01: rgb_comb = 3'b100;
32         2'b10: rgb_comb = 3'b010;
33         2'b11: rgb_comb = 3'b001;
34         default: rgb_comb = 3'b000;
35     endcase
36 end
37
38 endmodule

```

Figure 5: `comb_led_logic.sv` in the editor.

For example, `src/comb_led_logic.sv` contains some combinational logic examples that drive the single-colour LEDs in combinational mode:

```

led_comb[1] = sw[0] & sw[1];           // ?
led_comb[2] = sw[0] ^ sw[1];           // ?
led_comb[3] = sw[2] ? sw[3] : sw[0];   // ?
led_comb[7] = &sw;                     // 4-input reduction AND

```

Checkpoint — answer in the report worksheet

Checkpoint 1a. Which line is an **AND** gate, which is an **XOR** gate, and which is a **multi-plexer**?

Step 1.2 — Make the combinational testbench the simulation top.

GUI Sources → Simulation Sources → sim_1 → right-click tb_comb_led_logic → **Set as Top**.

Tcl Console (*alternative*)

```

set_property top tb_comb_led_logic [get_filesets sim_1]
update_compile_order -fileset sim_1

```

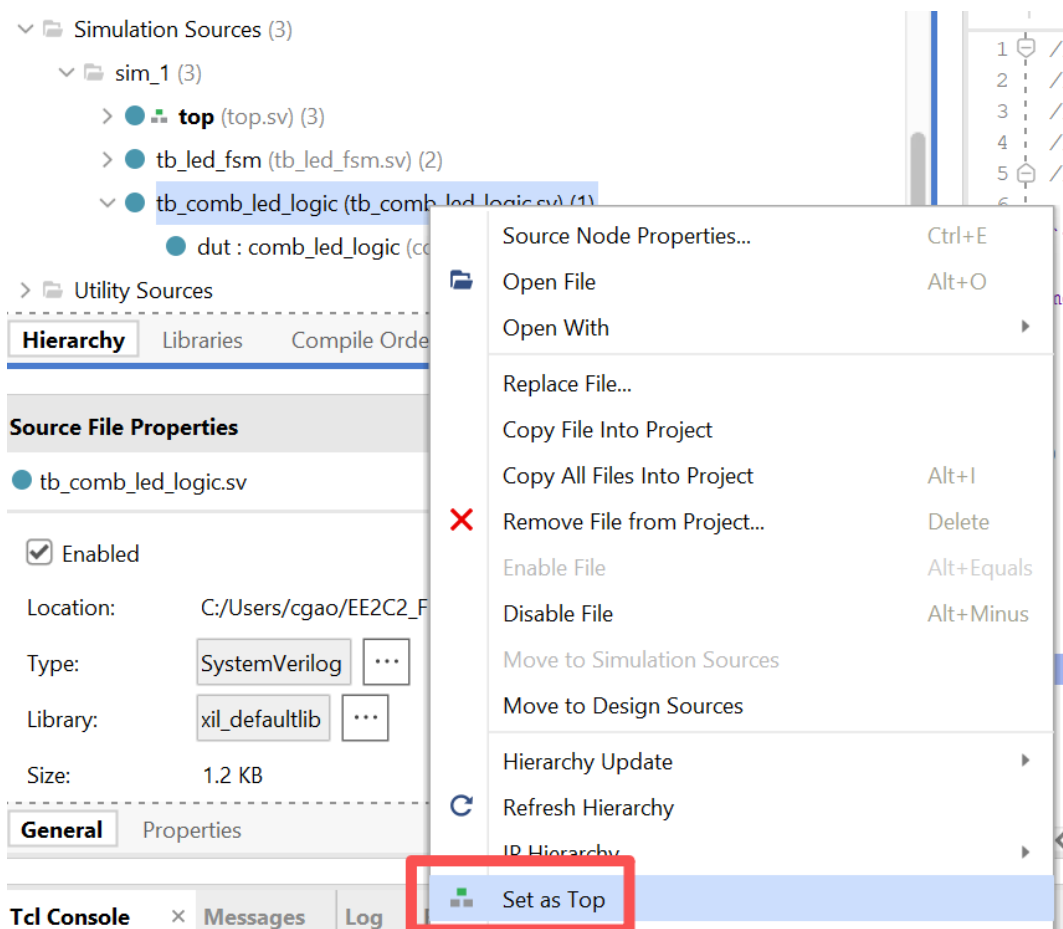


Figure 6: Setting the simulation top.

Step 1.3 — Run the behavioural simulation.**GUI** Flow Navigator → Run Simulation → Run Behavioral Simulation.**Note**

Close the previous simulation before launching another one. If a waveform/simulator window is still open, run `close_sim` in the Tcl Console (or close the current simulation from the GUI) before the next `launch_simulation`. Otherwise Vivado may report **ERROR: [Vivado 12-1501] Simulator for snapshot ... is already running.**

Tcl Console (alternative)

```
catch {close_sim} ;# harmless if no simulation is open
launch_simulation
run all
```

PowerShell (alternative, no GUI)

```
vivado -mode batch -source scripts\run_simulation.tcl -tclargs tb_comb_led_logic
```

The Tcl console should print lines like `sw=1111 btn=0 led_comb=...` ending with `tb_comb_led_logic completed`.

Step 1.4 — Read the waveform.

GUI Ensure `sw`, `btn`, `led_comb`, `rgb_comb` are visible; drag in any missing signal and click **Zoom Fit**. Observe that when `sw` changes, the outputs change at the *same* simulation time — there is no clock here.

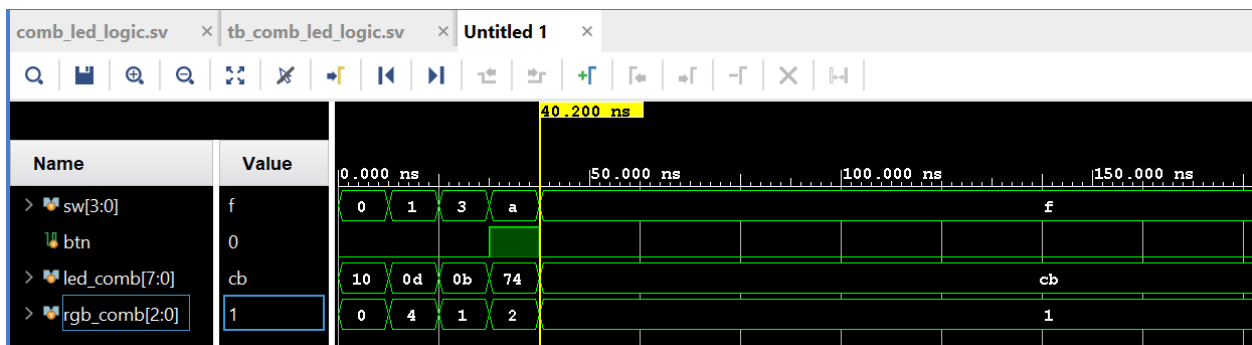


Figure 7: Behavioural-simulation waveform for the combinational logic.

Report evidence: capture the `tb_comb_led_logic` waveform and mark one input change where the output changes at the same simulation time.

Checkpoint — answer in the report worksheet

Checkpoint 1b. Why can a *testbench* change inputs with `#10` delays, while *synthesizable* RTL must not use `#10` delays?

3 Topic 2 — Sequential logic

3.1 Background

Sequential logic has memory: outputs can depend on past inputs. State is stored in **flip-flops** that update only on a **clock edge**.

- `always_ff @(posedge clk)` describes flip-flops.
- `<=` (non-blocking) is used for clocked updates.
- A **counter** is the simplest example.

```
always_ff @(posedge clk) begin
    if (rst) count <= '0;
    else     count <= count + 1'b1;
end
```

The lab module `src/one_hz_tick.sv` counts clock cycles and, on the wrap cycle, raises `tick_1s` for exactly one clock cycle. In the AC701 implementation, `top_ac701.sv` sets the counter limit for the 200 MHz board clock.

Note

A **tick** is a **clock enable**, not a new clock. `tick_1s` is a normal signal that is high for one cycle. The whole design still runs on the single 100 MHz `clk`.

3.2 Lab 2: inspect the counter and watch it tick

Step 2.1 — Open the source.

GUI Sources → Design Sources → double-click `one_hz_tick`. Find the `always_ff` block, the register `count_value`, and the line that pulses `tick_1s`.

Step 2.2 — Observe the counter and tick in simulation.

The counter runs together with the FSM in `tb_led_fsm`, which uses a tiny `SIM_CLK_FREQ_HZ = 8` so a tick happens every 8 cycles. (Full run in Topic 3.) **Tcl Console**

```
catch {close_sim} ;# close the Topic 1 simulation if it is still open
set_property top tb_led_fsm [get_filesets sim_1]
update_compile_order -fileset sim_1
launch_simulation
run all
```

Watch `count_value` increment $0 \rightarrow 7$ and `tick_1s` pulse for one cycle on each wrap.

Report evidence: capture a zoom of `count_value` wrapping and the one-cycle `tick_1s` pulse.

Checkpoint — answer in the report worksheet

Checkpoint 2. What hardware does `always_ff @(posedge clk)` infer, and why is `tick_1s` a clock-enable pulse rather than a new clock?

```

8 module one_hz_tick #(
9     parameter int CLK_FREQ_HZ = 100_000_000,
10    parameter int COUNTER_WIDTH = (CLK_FREQ_HZ <= 1) ? 1 : $clog2(CLK_FREQ_HZ)
11 ) (
12     input logic          clk,
13     input logic          rst,
14     output logic         tick_1s,
15     output logic [COUNTER_WIDTH-1:0] count_value
16 );
17
18     localparam logic [COUNTER_WIDTH-1:0] TERMINAL_COUNT_VALUE = CLK_FREQ_HZ - 1;
19
20     always_ff @(posedge clk) begin
21         if (rst) begin
22             count_value <= '0;
23             tick_1s <= 1'b0;
24         end else if (count_value == TERMINAL_COUNT_VALUE) begin
25             count_value <= '0;
26             tick_1s <= 1'b1;
27         end else begin
28             count_value <= count_value + {{(COUNTER_WIDTH-1){1'b0}}, 1'b1};
29             tick_1s <= 1'b0;
30         end
31     end
32
33 endmodule
34

```

Figure 8: one_hz_tick.sv in the editor.

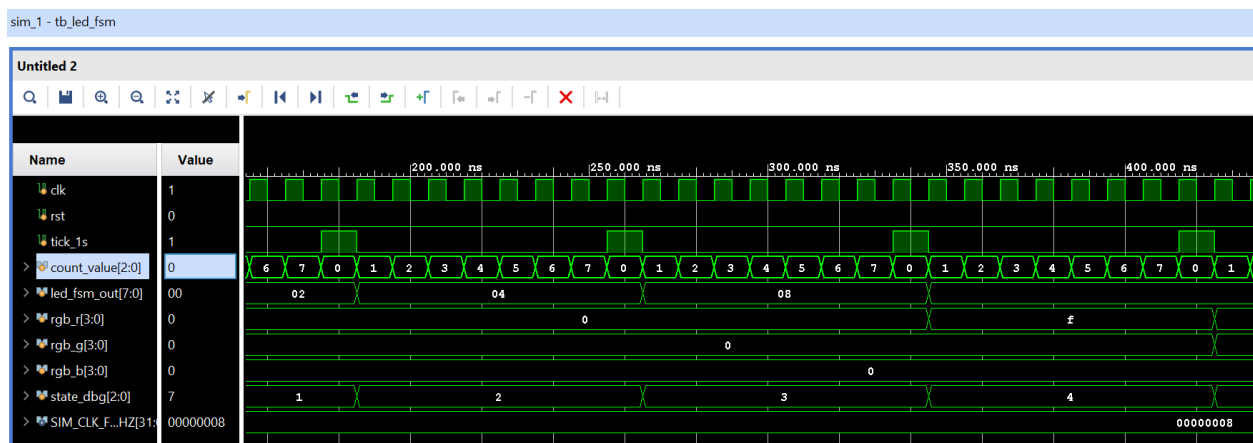


Figure 9: Counter and tick in the FSM testbench.

4 Topic 3 — Finite state machines

4.1 Background

A **finite state machine (FSM)** is sequential logic that moves through defined **states**. It has three parts: a **state register** (flip-flops, *sequential*), **next-state logic** (*combinational*), and **output decode logic** (*combinational*). The lab FSM (`src/led_fsm.sv`) advances one step per tick:

```
IDLE -> MONO_LED_0 -> MONO_LED_1 -> MONO_LED_2
      -> RGB_RED -> RGB_GREEN -> RGB_BLUE -> ALL_OFF -> (back to IDLE)
```

A transition happens *only when* `tick_1s` is high — once per second on the board.

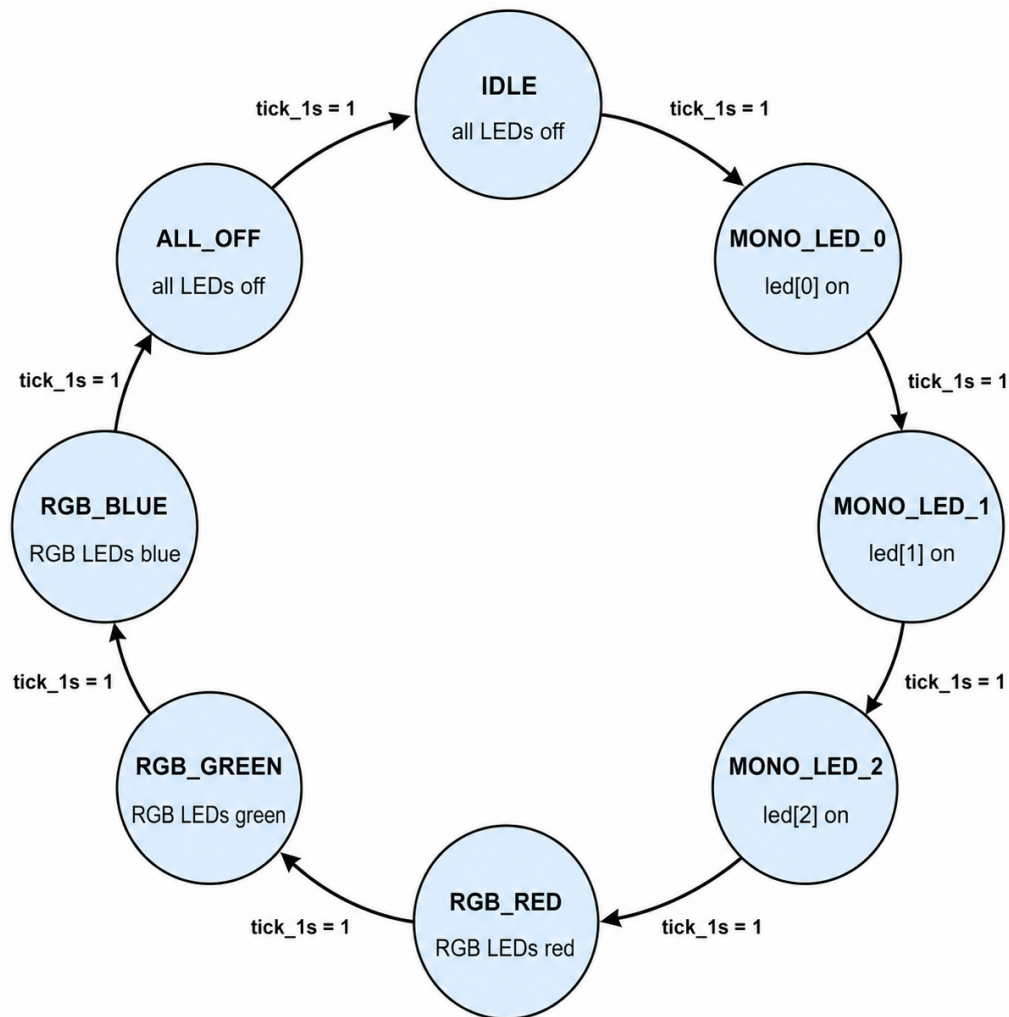


Figure 10: FSM state diagram.

4.2 Lab 3: inspect and simulate the FSM

Step 3.1 — Open and identify the three FSM parts.

GUI Sources → Design Sources → double-click `led_fsm`. Locate (1) the typedef enum state type, (2) the state register `always_ff ...state <= next_state;`, (3) the next-state `always_comb` guarded by `if (tick_1s)`, and (4) the output-decode `always_comb` that sets `led_fsm`, `rgb_r/g/b`.

Step 3.2 — Run the FSM testbench.

GUI Make `tb_led_fsm` the sim top (right-click → **Set as Top**), then **Flow Navigator** → **Run Simulation** → **Run Behavioral Simulation**.

PowerShell (alternative)

```
vivado -mode batch -source scripts\run_simulation.tcl -tclargs tb_led_fsm
```

Tcl Console (alternative)

```
catch {close_sim} ;# close any previous simulation snapshot first
set_property top tb_led_fsm [get_filesets sim_1]
update_compile_order -fileset sim_1
launch_simulation
run all
```

Step 3.3 — Read the FSM waveform.

GUI Add `clk`, `rst`, `tick_1s`, `count_value`, `state_dbg`, `led_fsm_out`, `rgb_r`, `rgb_g`, `rgb_b`; **Zoom Fit**. Observe: reset holds `IDLE`; after release, `tick_1s` pulses every 8 clocks; `state_dbg` advances one state per tick; outputs follow the state. The console prints one line per tick, ending with `tb_led_fsm completed`.

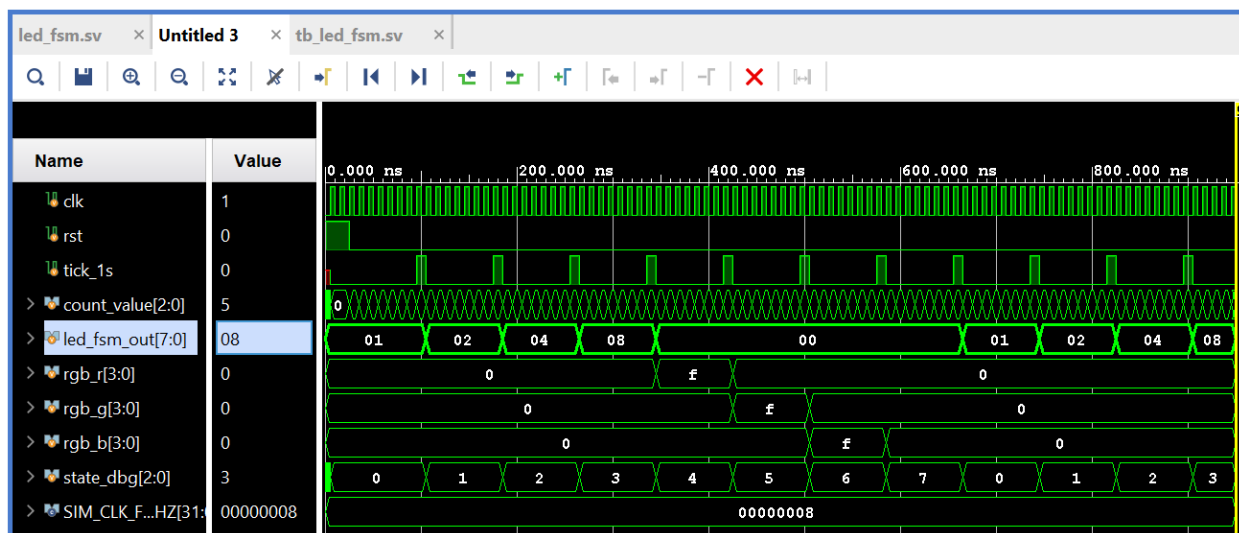


Figure 11: The FSM testbench waveform.

Report evidence: capture the `tb_led_fsm` waveform and mark the reset region plus two consecutive `tick_1s` pulses and their `state_dbg` transitions.

Checkpoint — answer in the report worksheet

Checkpoint 3. Using the FSM waveform, choose two consecutive `tick_1s` pulses after reset. For each pulse, record the state just before the pulse, the state just after the pulse, and the LED/RGB output after the transition. Then explain what this shows about the roles of the state register, next-state logic, and output-decode logic.

5 Topic 4 — From RTL to bitstream: synthesis and implementation

5.1 Background

The AC701 top `src/top_ac701.sv` converts the differential clock with an `IBUFDS`, synchronises `rst_btn`, instantiates the lab modules, and multiplexes between the FSM and combinational LED outputs on `sw[3]`. To reach a configured chip: **Synthesis** (RTL → LUT/FF netlist), **Implementation** (place & route onto this exact device), and **Bitstream generation** (`top_ac701.bit`).

5.2 Lab 4: build the design

Note

Do not run both methods at once. `build.tcl` deletes and recreates `build\`. If the GUI has the project open, Windows blocks the delete. **Close the GUI before running `build.tcl`**, or use the GUI buttons instead.

Option A — one scripted build.

PowerShell (close the GUI first)

```
vivado -mode batch -source build.tcl
```

Watch for INFO: Bitstream generated at: `...\impl_1\top_ac701.bit`.

Option B — click through the flow.

GUI Flow Navigator → Run Synthesis (OK); when it finishes choose Run Implementation (OK); then Generate Bitstream (OK).

Tcl Console (*equivalent*)

```
launch_runs synth_1 -jobs 4
wait_on_run synth_1
launch_runs impl_1 -to_step write_bitstream -jobs 4
wait_on_run impl_1
```

Step 4.1 — Confirm it built and check for errors.

GUI Open the **Design Runs** tab; `synth_1` and `impl_1` should show green check marks. Open **Messages** and confirm no red errors.

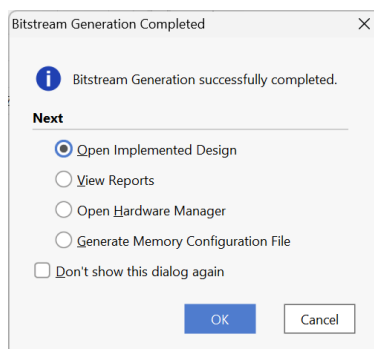


Figure 12: Design Runs after a successful build. Open the implemented design to see reports.

Checkpoint — answer in the report worksheet

Checkpoint 4. In one sentence, what is the difference between synthesis and implementation?

6 Topic 5 — Hardware targets and AUP-ZU3 connection

6.1 Background

The required implementation reports in this lab use the **AC701** project you built in Topic 4. Your bench board is the **RealDigital AUP-ZU3-4GB**. Those are different FPGA families, so the `top_ac701.bit` file from Topic 4 must **not** be programmed onto the AUP-ZU3.

To keep the lab portable on university PCs, the AUP-ZU3 parts of the lab use **prebuilt AUP-ZU3 bitstreams**. Vivado only needs Hardware Manager to download those files over JTAG; it does not need to synthesize, implement, or even create a project for the XCZU3EG device.

6.2 Lab 5: prepare the AUP-ZU3 for prebuilt demos

Step 5.1 — Connect and power the AUP-ZU3.

The board has **two USB-C ports**: **PROG UART** goes to the **PC** for JTAG/programming, and **EXT PWR** goes to a **USB-C PD power adapter**. The PROG UART cable alone does not power the board.

With the board **off**:

1. set the **BOOT** switch next to the microSD slot to **JTAG**, not SD;
2. connect the USB-A-to-USB-C data cable from the **PC** to **PROG UART**;
3. connect the USB-C PD adapter to **EXT PWR**;
4. power the board on and confirm the power LED turns on.

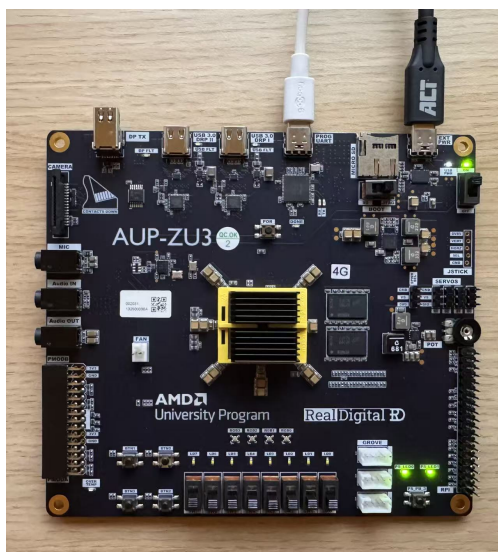


Figure 13: The powered AUP-ZU3 board with both USB-C cables attached.

Step 5.2 — Check that Hardware Manager can see the board.

Preflight before any prebuilt demo: **EXT PWR** is connected and the power LED is on; **BOOT** is set to **USB-JTAG/JTAG** (power-cycle after changing it); **PROG UART** is connected

with a data cable; and no other Vivado, Vitis, Hardware Manager, or `hw_server` session is already using JTAG.

GUI In Vivado, **Open Hardware Manager** → **Open Target** → **Auto Connect**. The device list should contain the FPGA device, usually named `xczu3_0` or `xczu3eg`. You may also see `arm_dap_1`; do not program that entry because it is the debug access port for the ARM Processing System, not the FPGA fabric.

Tcl Console The same check from a Tcl Console is:

```
open_hw_manager
connect_hw_server
open_hw_target
get_hw_devices
close_hw_target
close_hw_manager

***** Xilinx cs_server v2023.2.0
***** Build date   : Oct 10 2023-06:01:56
***** Build number : 2023.2.1696910516
***** Copyright 2017-2022 Xilinx, Inc. All Rights Reserved.
***** Copyright 2022-2026 Advanced Micro Devices, Inc. All Rights Reserved.

connect_hw_server: Time (s): cpu = 00:00:01 ; elapsed = 00:00:09 . Memory (MB): peak = 3643.414 ; gain = 1.332
localhost:3121
open_hw_target
INFO: [Labtoolstcl 44-466] Opening hw_target localhost:3121/xilinx_tcf/Xilinx/88022500036AA
get_hw_devices
xczu3_0 arm_dap_1
close_hw_target
INFO: [Labtoolstcl 44-464] Closing hw_target localhost:3121/xilinx_tcf/Xilinx/88022500036AA
close_hw_manager
```

Figure 14: The AUP-ZU3 detected in Hardware Manager.

Note

Use only one JTAG connection at a time. Close Hardware Manager before running a programming script, or let the script open Hardware Manager itself. If a script says the target is busy, close other Vivado or Vitis windows and retry.

Checkpoint — answer in the report worksheet

Checkpoint 5. Why do Topics 4, 6, and 7 use the AC701 project, while the AUP-ZU3 board demos use prebuilt bitstreams? In one sentence, also state why `top_ac701.bit` must not be programmed onto the AUP-ZU3.

Troubleshooting

If **Auto Connect** finds nothing, or a prebuilt demo script reports that no FPGA device was found:

- **Power** — is **EXT PWR** plugged in and the power LED on? **PROG UART** alone does not power the board.
- **Boot mode** — is the **BOOT** switch on **JTAG** (not SD)? Power-cycle after changing it.

- **JTAG busy** — close any other Vivado/Vitis or running script holding the target.
- **Wrong target** — program the xczu3/xczu3eg FPGA device, not arm_dap.

7 Topic 6 — Static timing analysis

7.1 Background

In the AC701 implementation the board clock is 200 MHz, so the clock period is 5 ns. Every signal launched by one flip-flop on a clock edge must arrive at the capturing flip-flop, and settle, *before* the next edge. Static timing analysis (STA) checks this for *all* register-to-register paths without simulating any input vectors.

Setup constraint (data must not arrive too late).

$$T_{\text{clk}} \geq T_{\text{clk-to-q}} + T_{\text{comb}}^{\text{max}} + T_{\text{setup}} + \text{margin}.$$

The longest combinational path sets the maximum usable clock frequency: if the data arrives after the setup window opens, the captured value is undefined.

Hold constraint (data must not arrive too early).

$$T_{\text{clk-to-q}} + T_{\text{comb}}^{\text{min}} \geq T_{\text{hold}} + T_{\text{skew}}.$$

The *shortest* path must be long enough that the new value does not race through and overwrite the data before the capturing flip-flop has latched the *previous* value. Note this is independent of clock frequency. A hold violation cannot be fixed by slowing the clock.

Slack.

$$\text{slack} = T_{\text{required}} - T_{\text{arrival}}.$$

Positive slack means the path meets timing with margin to spare; negative slack means it fails. Vivado reports the following:

- **WNS** — Worst Negative Slack: the slack of the single tightest setup path. $\text{WNS} \geq 0$ means *every* setup path is met.
- **TNS** — Total Negative Slack: the sum of all negative setup slacks. $\text{TNS} = 0$ confirms there are no setup violations.
- **WHS** / **THS** — Worst / Total Hold Slack, the hold-side equivalents of WNS / TNS.
- **Critical path** — the path with the smallest slack, i.e. the one closest to failing (and the first to fix if timing is not met).

In this small design the critical path usually lies inside the one-second counter, where the wide comparator and adder form the longest combinational chain.

7.2 Lab 6: read the timing report and find the critical path

Step 6.1 — Open the implemented design.

GUI Flow Navigator → Open Implemented Design.

Tcl Console open_run impl_1

Step 6.2 — Open the timing summary.

GUI Reports → Timing → Report Timing Summary (OK).

Tcl Console report_timing_summary

Step 6.3 — Inspect the worst (critical) path.

GUI In the **Timing** tab expand **Setup** → your clock and click the first (worst) path. The **Path Properties** show Source (From), Destination (to), Source Clock, Destination Clock, Total Delay, Logic Delay and Net Delay. Record them.

Step 6.4 — See the path as hardware.

GUI Right-click the selected path → **Schematic**. Identify the launch flip-flop, the combinational logic (LUTs / carry chain), and the capture flip-flop.

Report evidence: capture the timing summary and/or the critical-path schematic with the launch flip-flop, logic, and capture flip-flop labelled.

Checkpoint — answer in the report worksheet

Checkpoint 6. Record WNS and TNS; state whether timing is met and how you know; record the critical path start/endpoint; and describe the path in one sentence with a screenshot of the schematic attached.

8 Topic 7 — Power analysis

8.1 Background

Vivado's power report *estimates* on-chip power and splits it into two parts: **static** power (transistor leakage, present whenever the device is powered) and **dynamic** power (drawn only when nodes switch). Vivado further breaks the dynamic part into **clock**, **logic/signal**, and **I/O** power.

Dynamic switching power follows

$$P_{\text{dyn}} \approx \alpha C V_{\text{DD}}^2 f,$$

where α is the switching (toggle) activity, C the switched capacitance, V_{DD} the supply voltage, and f the clock frequency. The activity α is the key unknown: unless you supply real switching data (e.g. a SAIF file from simulation), Vivado performs a *vectorless* estimate using assumed default toggle rates, and flags this with a low **Confidence level**. The V_{DD}^2 term is why supply voltage dominates the energy budget, and the linear f term is why the clock and clock-driven logic are often the largest dynamic contributors.

8.2 Lab 7: read the power report

Step 7.1 — Open the power report.

GUI Reports → Report Power (OK).
Tcl Console report_power

Step 7.2 — Record the numbers.

From the **Summary**, record the total on-chip power, the static and dynamic totals, and the clock / logic-signal / I/O components; note which is largest. Also record the reported **Confidence level**, as it tells you how much to trust the estimate.

Report evidence: capture the Power Summary panel with total power, static/dynamic split, and the largest category visible.

Checkpoint — answer in the report worksheet

Checkpoint 7.

1. Which power category is largest, and does the $\alpha C V_{\text{DD}}^2 f$ relation explain why?
2. What assumptions drive the estimate, and what would you provide to improve its confidence level?
3. Does this report include the power consumed by the physical LEDs on the board?

9 Topic 8 — PWM DAC: the digital/analog boundary

9.1 Background

Pulse-Width Modulation (PWM) encodes an analog level as the fraction of time a digital signal is high (the **duty cycle**). A free-running counter compares against **duty**: $\text{pwm_out} = (\text{count} < \text{duty})$, so **pwm_out** is high for duty of every 2^N cycles.

- With $N = 8$, one period is $2^8 = 256$ cycles, so the PWM frequency is $100 \text{ MHz}/256 = 390.625 \text{ kHz}$.
- The selector maps $\text{sw}[3:0]$ to $\text{duty} = \{\text{sw}, 4'b0000\}$: $0000 \rightarrow 0\%$, $1111 \rightarrow 240/256 \approx 93.75\%$ (this simple version never reaches true 100%).

An LED driven by PWM looks dimmer/brighter because the eye averages the fast on/off. In this lab we keep the hardware simple and drive only the on-board LED. The same PWM idea can be filtered in other designs, but there is no PMOD output in this version.

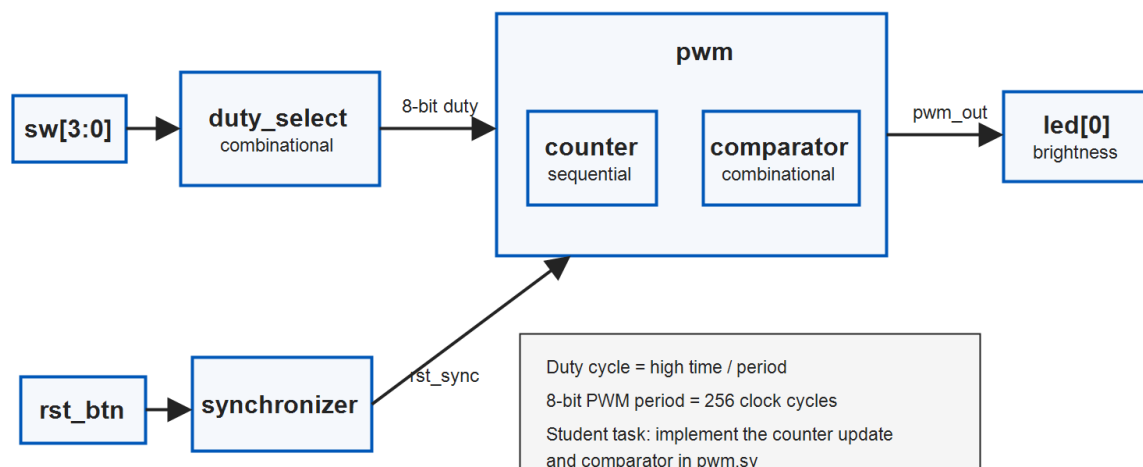


Figure 15: The PWM data path.

9.2 Lab 8: complete, simulate, and optionally observe the PWM LED dimmer

Step 8.1 — Complete the PWM mechanism.

Skim the PWM source files:

- `pwm_dac/src/duty_select.sv`: combinational duty selection.
- `pwm_dac/src/pwm.sv`: sequential counter plus combinational comparator.
- `pwm_dac/src/synchronizer.sv`: reset-button synchronizer.
- `pwm_dac/src/top_pwm_dac.sv`: top-level wiring.

Then open `pwm_dac/src/pwm.sv` and fill in the two TODO lines:

1. update the N-bit counter once per clock when reset is not asserted;
2. drive `pwm_out` high when `count < duty`.

Step 8.2 — Simulate the PWM generator.

`pwm_dac/sim/tb_pwm.sv` drives several duty values and checks `pwm_out` is high for exactly duty cycles per 256-cycle period. Run it only after completing `pwm.sv`.

PowerShell Run the scripted simulation:

```
vivado -mode batch -source pwm_dac\scripts\run_pwm_simulation.tcl
```

If the TODO lines are correct, the console and `pwm_dac/build_pwm_dac/xsim_pwm/xsim.log` will include a clear pass line:

```
PWM_STUDENT_TEST_PASS: all 7 duty-cycle checks passed.
```

If it fails, read the first [FAIL] or compile error, fix `pwm_dac/src/pwm.sv`, and rerun.

Before answering the Topic 8 worksheet, show your responsible TA the PWM_STUDENT_TEST_PASS line. This is the required check for your own PWM implementation.

Report evidence: capture the console or `xsim.log` line showing PWM_STUDENT_TEST_PASS.

Step 8.3 — Optional prebuilt AUP-ZU3 brightness demo.

The university PC may not be able to implement an XCZU3EG design, so the board demonstration uses a prebuilt AUP-ZU3 bitstream from `prebuilt_bitstreams/`. This demo shows the expected LED brightness behavior; it is not proof that your TODO lines are correct. Your required proof is the passing testbench from Step 8.2.

PowerShell After the AUP-ZU3 connection preflight in Topic 5, program the prebuilt demo:

```
vivado -mode batch -source pwm_dac\scripts\program_prebuilt_pwm_jtag.tcl
```

Set `sw[3:0]` low (e.g. 0001) — `led[0]` is dim; raise toward 1111 — brighter. `led[4:1]` mirrors your switch code.

Checkpoint — answer in the report worksheet

Checkpoint 8.

1. For a 100 MHz clock and 8-bit counter, what is the PWM frequency (show the arithmetic)?
2. Why does changing duty change LED brightness?
3. Which TODO line describes sequential logic, and which describes combinational logic?
4. Why does `sw[3:0]=1111` not produce a true 100% duty cycle in this selector?

10 Topic 9 — Optional rainbow RTL demo

10.1 Just for fun

If your group finishes early, you can program a prebuilt rainbow demo for the AUP-ZU3. It drives the four RGB LEDs through a slowly changing hue generator. The switches still mirror onto the single-colour LEDs, and the pushbuttons change the rainbow speed, direction, and phase spacing.

Step 9.1 — Program the prebuilt rainbow demo.

Keep the same AUP-ZU3 connection and JTAG preflight from Topic 5, then program the prebuilt rainbow bitstream:

```
vivado -mode batch -source rainbow/scripts/program_prebuilt_rainbow_jtag.tcl
```

Step 9.2 — Play with the controls.

- `sw[7:0]` mirror directly onto `led[7:0]`.
- Button 0 makes the colour cycle faster.
- Button 1 makes the colour cycle slower.
- Button 2 reverses the colour direction.
- Button 3 changes the phase spacing between RGB LEDs.

Note

Topic 9 is optional and has no report worksheet questions. It is a small reward lap after the required PWM testbench pass is checked by your TA.

Lab completion checklist

Your group passes the full lab flow if you ran at least one behavioural simulation, completed the AC701 implementation, recorded timing and power report values, identified a critical path, completed the PWM TODOs, showed the `PWM_STUDENT_TEST_PASS` line to your responsible TA, and answered the checkpoint questions with reasonable understanding. The prebuilt AUP-ZU3 PWM and rainbow bitstreams are for demonstration only.

Note

Board return is required. Your group can only pass the lab after returning the FPGA board box to the responsible TA.

A Command quick reference

Run PowerShell commands from the **repository root** (the `EE2C2_FPGA_Lab` folder you cloned) after adding Vivado to `$env:Path` in Topic 0.

Task	Command
Clone the lab	<code>git clone https://github.com/lab-emi/EE2C2_FPGA_Lab.git</code>
Create project (no build)	<code>vivado -mode batch -source scripts\create_vivado_project.tcl</code>
Open project in GUI	<code>vivado build\EE2C2_FPGA_Lab.xpr</code>
Combinational sim	<code>vivado -mode batch -source scripts\run_simulation.tcl -tclargs tb_comb_led_logic</code>
FSM sim	<code>vivado -mode batch -source scripts\run_simulation.tcl -tclargs tb_led_fsm</code>
Build AC701 (synth+impl+bit)	<code>vivado -mode batch -source build.tcl</code>
Open implemented design	<code>(Tcl) open_run impl_1</code>
Timing summary	<code>(Tcl) report_timing_summary</code>
Power report	<code>(Tcl) report_power</code>
Run PWM DAC sim	<code>vivado -mode batch -source pwm_dac\scripts\run_pwm_simulation.tcl</code>
Program prebuilt PWM demo	<code>vivado -mode batch -source pwm_dac\scripts\program_prebuilt_pwm_jtag.tcl</code>
Program prebuilt rainbow demo	<code>vivado -mode batch -source rainbow\scripts\program_prebuilt_rainbow_jtag.tcl</code>

B Troubleshooting

- `vivado` not recognised — from the repository root, run:

```
$env:Path += ";C:\Xilinx\Vivado\2023.2\bin"
$env:Path += ";C:\Xilinx\Vitis\2023.2\bin"
vivado -version
```

If Vivado is installed somewhere else, replace the path with that installation's `bin` folder.

- Specified part could not be found for `xczu3eg-sfvc784` — you ran an old AUP-ZU3 build script. Use `build.tcl` for the required AC701 implementation, and use the prebuilt programming scripts for the AUP-ZU3 demos.
- `build.tcl` cannot delete `build\` — the GUI has the project open; close Vivado and retry.
- Synthesis red error — read the first **Messages** error; look for a missing semicolon, a typo, or a port mismatch.
- Board not detected — check the USB-JTAG cable and power; ensure no other Vivado holds the JTAG target.
- Wrong sources in the project — re-run `create_vivado_project.tcl` to rebuild cleanly.